

Executing a Policy End to End

An epistemic test in `plansys2_tests`, and the first tests for `plansys2_epistemic_bt_builder`

ePlanSys

21 August 2026

Abstract

The epistemic stack could produce a branching policy and could render one as a behavior tree, but nothing had ever run one over a live ROS graph, and the package that turns a policy into a tree had no tests and was excluded from coverage. This report describes the two end-to-end tests now in `plansys2_tests` — one corridor and one depot mission, each run twice with the world in different states — the twelve-test suite added to `plansys2_epistemic_bt_builder`, and the four defects that had to be fixed before any of it could pass. All four were latent: each one is invisible to a unit test and fatal to a real execution.

1 What was asked

Write the epistemic end-to-end test in `plansys2_tests`: drive a branching policy through `EpistemicBTBuilder` and the `EpistemicSwitch` node over a real ROS graph, using the multi-pointed `robot-fleet` fixtures. Alongside it, give `plansys2_epistemic_bt_builder` the tests it never had, and take it out of the coverage ignore list it sits in.

2 What was already there

The pieces existed and were individually tested:

- `plansys2_epistemic_planner` solves a grounded epistemic task with AO* and, under `conditional_plan: policy`, returns a `plansys2_msgs/Plan` whose items carry the branch structure (`children`, `outcomes`, `sensing`, `knowledge_requirements`, `epistemic_action`).
- `plansys2_epistemic_executor` holds the reading of those fields (`Policy`), the rendering of a policy as a tree (`policy_to_bt`), the four behavior tree nodes, and the `epistemic_state` node that answers knowledge questions and performs the DEL update.
- `plansys2_epistemic_bt_builder` is the `BTBuilder` plugin that joins the two — and had no tests at all.

What was missing was the join itself: nobody had ever asked a real `ExecutorNode` to load that builder, render a real policy, and run it against real action performers.

3 Four defects the end-to-end path exposed

Each of these is fixed in this change set. None of them could have been found by testing a component on its own.

1. A builder loaded by name was never initialised

`ExecutorNode::get_tree_from_plan` called `initialize()` only for `SimpleBTBuilder` and `STNBTBuilder`. Any other builder — which is to say, any builder loaded through the `bt_builder_plugin` parameter that the plugin mechanism exists to support — was constructed and used uninitialised. The configured action template and time precision were silently discarded. Fixed by initialising the unrecognised case as well.

2. A template that cannot hold a policy was accepted

With (1) fixed, the executor now hands the epistemic builder whatever action template it is configured with, and its default is PlanSys2's own — which has no `CONTINUATIONS` placeholder. Rendering a policy into it produces a tree holding only the root action, which runs, succeeds, and leaves the mission undone: the worst kind of failure, because it reports success. `EpistemicBTBuilder::initialize` now refuses such a template, warns, and falls back to the packaged one.

3. An empty policy rendered an unparseable tree

A planner handed a goal that already holds returns an empty plan. The rendering of an empty policy with no epistemic goal was `<Sequence name="EpistemicPolicy"/>` with no children, which `BehaviorTree.CPP` rejects outright — so “there is nothing to do” crashed tree creation. `policy_to_bt` now emits `<AlwaysSuccess/>` in that case.

4. A missing exported dependency

`plansys2_epistemic_planner` and `plansys2_epistemic_executor` carry `nlohmann_json` in their exported targets' link interfaces but did not name it in `ament_export_dependencies`. Nothing noticed while every consumer happened to `find_package` it for its own reasons; `plansys2_tests` is the first that does not, and it failed to configure. Both packages now export it.

4 The gap that had to be filled: where an observation comes from

`ApplyEpistemicUpdate` takes the observed outcome on an `observed` port, and the packaged template leaves it unbound — deliberately, since what a robot saw is domain-specific and PlanSys2 carries nothing back from an action performer. The epistemic state can name the outcome itself only when its model already designates a single world, and both fleet fixtures designate several: that is what makes them worth testing with, and it means the observation must come from outside.

The test therefore supplies what a deployment would have to supply:

- `plansys2_tests/behavior_trees/observing_action_bt.xml` — the packaged epistemic template with one node added and `observed` bound.
- `ObservedOutcome` (`plansys2_tests_observation_bt_nodes`, loaded by the executor through `bt_node_plugins`) — a stand-in for a performer that reports: it reads `action=outcome` entries from a latched topic and reports the one matching its own action. Per action, because the depot has two things to find out and they need not agree.

5 What the tests do

plansys2_tests/test_5 — one corridor, one branch

The full graph is brought up in one process: domain expert, problem expert, planner (with the plansys2/EpistemicPlanSolver plugin pointed at robot-fleet.json and its action mapping), executor (with bt_builder_plugin: EpistemicBTBuilder and both node libraries), epistemic_state loaded from the same task, four fake action performers, and the execution logger. The classical half of the mission is test_5/pddl/test_5.pddl, which records nothing about the corridor: a predicate for “blocked” would be recording knowledge as a fact.

The planner returns a four-node policy. The same mission is then run twice, differing only in what the corridor turns out to be, and the two runs must execute different actions: report_clear in one and report_blocked in the other, with the branch not taken never running.

plansys2_tests/test_6 — two corridors, a branch inside a branch

The depot mission: three robots, two corridors, an initial model designating all four combinations of blocked and clear, and a twelve-node policy with a switch inside a switch’s continuation. The two runs report the corridors in opposite states and share no leaf.

plansys2_epistemic_bt_builder/test — twelve tests

The plugin’s contract with plansys2_executor: that a branching policy renders one switch with its outcomes in order and both continuations present; that a plan with no branches renders as the flat sequence PlanSys2 would build; that a malformed policy is reported by returning no tree; that the action id written into the tree is exactly the one the executor keys its action map by (policy_action_id against BTBuilder::to_action_id); that the template refusal of defect (2) works; that the dotgraph is drawn from the policy, with the observation labelling each edge, the knowledge guard drawn with the node it guards, and colours taken from the executor’s action map; and that pluginlib can find the class under the name the executor asks for. Every rendered tree is handed to the real BehaviorTree.CPP parser.

6 Results

Verified in the ros:rolling container, the same environment .github/workflows/rolling.yaml builds in.

Suite	Result	Time
epistemic_bt_builder_test (12 tests)	all passed	0.16 s
test_5 (2 tests)	all passed	32.3 s
test_6 (2 tests)	all passed	56.6 s
plansys2_epistemic_executor (35 existing tests)	all passed	—
plansys2_tests (165 tests, lint included)	all passed	3 min 18 s

The interesting part is the log, which shows the model being narrowed by what was executed. From the depot run with north blocked and south clear:

```
[epistemic_state] applied goto-north_r1: 4 worlds, 4 designated
[epistemic_state] applied inspect-north_r1 -> e-inspect-north-blocked: 4 worlds, 2 designated
[epistemic_state] applied report-north-blocked_r1: 4 worlds, 2 designated
[epistemic_state] applied goto-south_r2: 4 worlds, 2 designated
[epistemic_state] applied inspect-south_r2 -> e-inspect-south-clear: 4 worlds, 1 designated
[epistemic_state] applied report-south-clear_r2: 4 worlds, 1 designated
[goal] (and (Kw r1 blocked-north) ... (Kw r3 blocked-south)) holds
```

Four candidate worlds at the start, two after the first sensing, one after the second, and the goal checked against the state that actually resulted rather than the one planning assumed.

7 Coverage

`plansys2_epistemic_bt_builder` is removed from the `packages-ignore` list in the `lcov-result` configuration of `.github/workflows/rolling.yaml`. That list exists for packages producing no `.gcda` at all; with a gtest suite the builder now produces one, and leaving it there would discard the coverage it generates.

8 Fixtures

The mission fixtures — `robot-fleet.json`, `robot-fleet-depot.json` and their action mappings — exist only on the unmerged branch `feature/epddl-front-end` (commit 58bb21b9). They are restored into their canonical paths in the working tree so the tests can run, but they are not this change's to introduce: when that branch merges, these files should come from it rather than from here. Nothing else in the change set depends on that branch.

9 Limitations

- The two missions are the two fixtures. A third case worth having is the negative one: an outcome the policy does not plan for, where `EpistemicSwitch` must fail rather than guess. It needs a performer that reports something the model cannot account for, and is not written.
- `ObservedOutcome` is a test's stand-in for a sensor. A deployment still has to bind `observed` to its own performers; nothing in the stack does it automatically, and that remains the one piece of wiring a domain must supply itself.
- The tests build only against ROS 2 rolling. The Humble CI job stops before `plansys2_executor`, as it did before this change.

10 Files

Added

- `plansys2_epistemic_bt_builder/test/` — the suite and its CMake.
- `plansys2_tests/test_5/`, `plansys2_tests/test_6/` — the two missions, their PDDL and their READMEs.
- `plansys2_tests/behavior_trees/observing_action_bt.xml`, `plansys2_tests/src/plansys2_tests/behavior_trees/observing_action_bt.xml`
- The four fixture files, restored (see above).

Changed

- `plansys2_executor/src/plansys2_executor/ExecutorNode.cpp` — defect (1).
- `plansys2_epistemic_bt_builder/src/epistemic_bt_builder.cpp` — defect (2).
- `plansys2_epistemic_executor/src/policy_bt.cpp` — defect (3).

- `plansys2_epistemic_planner/CMakeLists.txt`, `plansys2_epistemic_executor/CMakeLists.txt`
— defect (4).
- `plansys2_epistemic_bt_builder/{CMakeLists.txt,package.xml}`, `plansys2_tests/{CMakeLists.txt,package.xml}`
— build wiring.
- `.github/workflows/rolling.yaml` — coverage.